CS683: Advanced Computer Architecture

Programming Assignment 2 Comprehensive Report

Prefetching, Instruction-Conscious RRIP, and Synergy Analysis

Course: CS683 **Term:** Autumn 2026

October 15, 2026

Abstract

This report presents a comprehensive empirical evaluation of memory hierarchy optimizations on ChampSim across three SPEC benchmark traces. We examine three core architectural milestones: (1) **Task 1B: Stride Pattern Prefetcher (SPP)** at the L1 Data Cache featuring an accuracy-throttled dynamic degree mechanism; (2) **Task 2: Instruction-Conscious RRIP Replacement Policy (I-RRIP)** at the shared L2 Cache, evaluating both static prioritization (Task 2A) and dynamic set-dueling adaptation across a 5-threshold sensitivity sweep (Task 2B); and (3) **Task 3: Synergy of SPP and Dynamic I-RRIP**, evaluating prefetch-aware replacement handling to eliminate cache pollution while hiding memory latency.

---

1 Task 1B: Stride Pattern Prefetcher (SPP)

1.1 Experimental Setup

Task 1B evaluates the Stride Pattern Prefetcher (SPP) at the L1 Data Cache against a baseline no-prefetch configuration and a standard IP-stride prefetcher. Simulations were executed on ChampSim with a 25-million instruction warmup and a 25-million instruction region of interest per trace, using a single-core out-of-order processor model, hashed-perceptron branch predictor, and LRU replacement.

Prefetch accuracy and coverage follow standard assignment formulations:

$$\text{Coverage} = \frac{\text{Baseline L1D Load Misses} - \text{Prefetcher L1D Load Misses}}{\text{Baseline L1D Load Misses}} \times 100\% \tag{1}$$

$$\text{Accuracy} = \frac{\text{L1D Prefetch Hits}}{\text{Total L1D Prefetches Issued}} \times 100\% \tag{2}$$

1.2 Measured Results

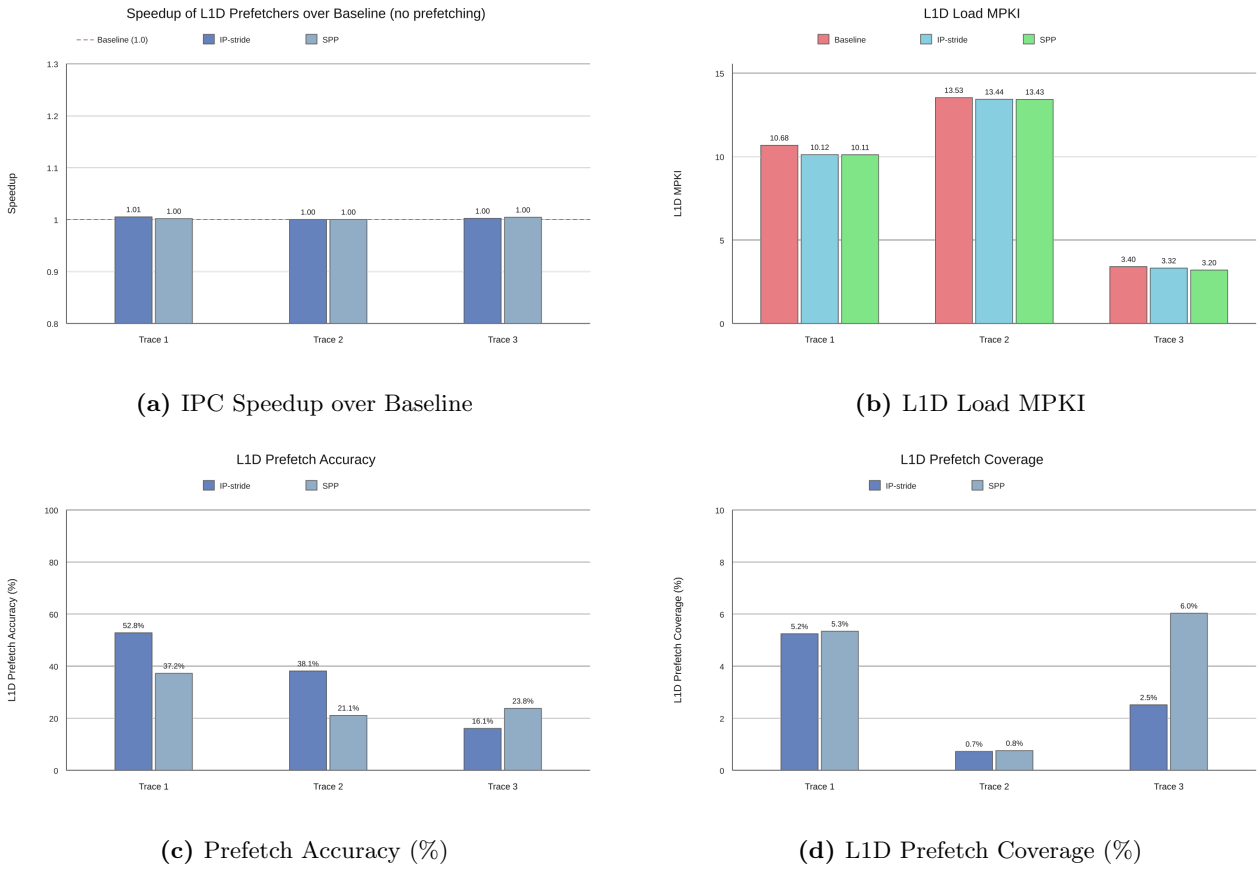
Table 1 summarizes the performance and cache metrics for each trace under the baseline, IP-stride, and SPP configurations.

Table 1: Measured IPC and prefetcher metrics for Task 1B across each trace.

Trace	Configuration	IPC	Speedup	L1D Load MPKI	Accuracy (%)	Coverage (%)
Trace 1	Baseline	0.232386	1.000000	10.6787	–	–
	IP-stride	0.233398	1.004355	10.1188	52.7914	5.2433
	SPP	0.232646	1.001119	10.1085	37.2389	5.3400
Trace 2	Baseline	0.250946	1.000000	13.5338	–	–
	IP-stride	0.251186	1.000956	13.4356	38.1276	0.7253
	SPP	0.251183	1.000944	13.4314	21.0828	0.7563
Trace 3	Baseline	0.382022	1.000000	3.4032	–	–
	IP-stride	0.382962	1.002461	3.3177	16.0936	2.5141
	SPP	0.383744	1.004508	3.1980	23.7845	6.0319

1.3 Plots

Figure 1 presents the empirical evaluation metrics for Task 1B across all three traces.



(a) IPC Speedup over Baseline

(b) L1D Load MPKI

(c) Prefetch Accuracy (%)

(d) L1D Prefetch Coverage (%)

Figure 1: Task 1B Metrics: Performance, MPKI, accuracy, and coverage across traces for Baseline, IP-stride, and SPP.

1.4 SPP Architecture and Exact Storage Overhead

The SPP tracks load stream behavior per instruction pointer using a 12-bit folded IP tag. The pattern table indexes 128 entries to learn recurring delta patterns.

Table 2: SPP table entries and hardware storage breakdown (including compiler alignment).

Table	Fields per Entry	Entries	Declared Bits	Allocated Bits
IP Tracker	valid (8), IP (16), page tag (16), line offset (8), last stride (8), uniform confidence (8), sig (16), LRU (8)	32	88	112
Pattern Table	next stride (8), confidence (8)	128	16	16
Prefetch Filter	valid (8), tag (16)	32	24	32
Total Tables			**5,632 bits**	**6,656 bits (832 B)**

Accounting for four 64-bit statistics counters and three 32-bit epoch/degree counters (352 bits), the total SPP hardware state is **876 bytes (7,008 bits)**.

1.5 Accuracy Throttling Rationale

Prefetch degree adapts dynamically after every epoch of 128 requests:

$$\text{Degree} = \begin{cases} 1, & \text{if Accuracy} < 25\% \\ 2, & \text{if } 25\% \leq \text{Accuracy} < 50\% \\ 4, & \text{if } 50\% \leq \text{Accuracy} < 75\% \\ 6, & \text{if Accuracy} \geq 75\% \end{cases} \tag{3}$$

Starting at degree 2 ensures sufficient sampling before adapting. Throttling suppresses useless prefetches in noisy, low-confidence regimes (limiting bus contention and cache pollution) while allowing aggressive lookahead in highly predictable sequences.

---

2 Task 2: Instruction-Conscious RRIP (I-RRIP) Policies

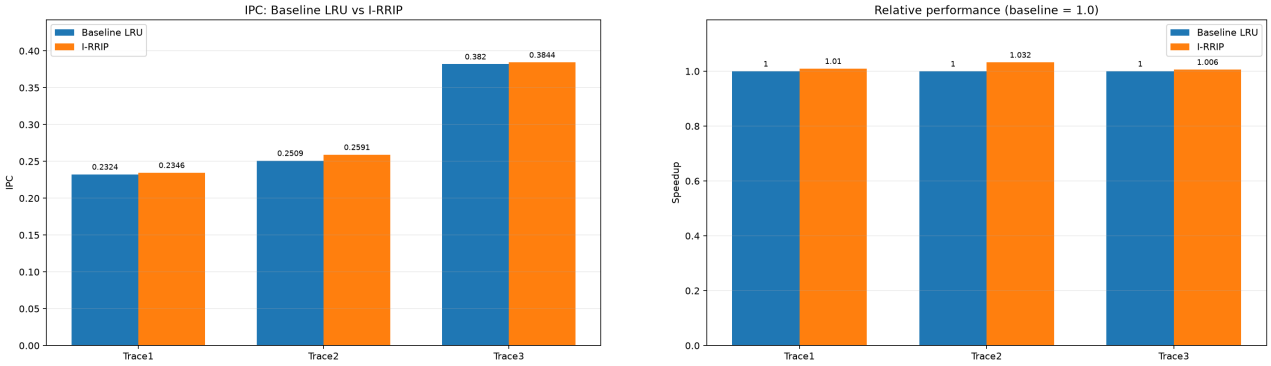
2.1 Task 2A: Baseline LRU vs. Static I-RRIP

Static I-RRIP protects instruction lines in the shared 8-way, 1024-set (512 KB) L2 cache against displacement by aggressive streaming data lines. Instruction lines are inserted at $RRPV = 0$ (immediate high priority), while data lines enter at $RRPV = 2$. Any hit promotes the line to $RRPV = 0$.

Table 3: Task 2A: Performance and L2 load MPKI comparison (Baseline LRU vs. Static I-RRIP).

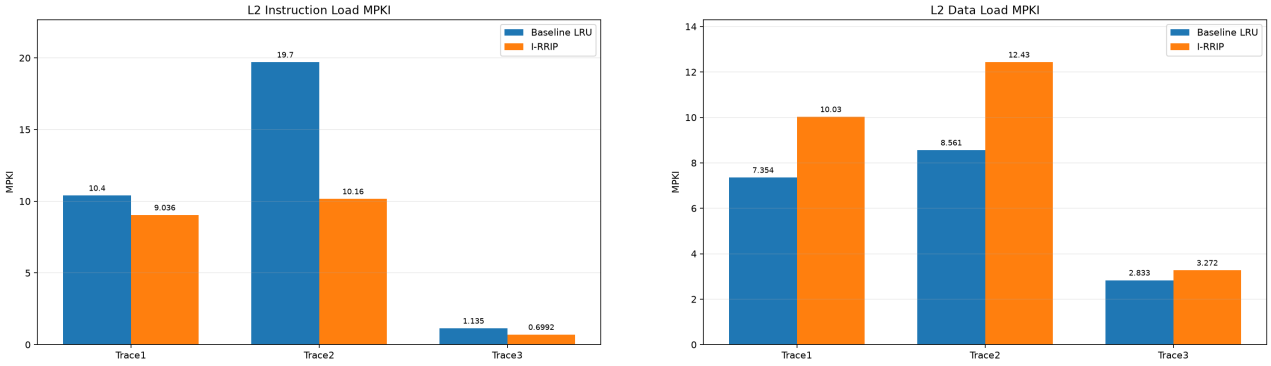
Trace	Policy	IPC	Speedup	L2C Load MPKI	Instruction MPKI	Data MPKI
Trace 1	Baseline LRU	0.232386	1.000000	17.7506	10.3965	7.3541
	Static I-RRIP	0.234612	1.009579	19.0635	9.0358	10.0277
Trace 2	Baseline LRU	0.250946	1.000000	28.2597	19.6990	8.5607
	Static I-RRIP	0.259057	1.032322	22.5938	10.1630	12.4308
Trace 3	Baseline LRU	0.382022	1.000000	3.9686	1.1354	2.8332
	Static I-RRIP	0.384399	1.006222	3.9708	0.6992	3.2716

Analysis: On Trace 2, Static I-RRIP yields a substantial 3.23% speedup by reducing instruction misses from 19.6990 to 10.1630 MPKI. However, because static prioritization is rigid, data lines are prematurely evicted, causing data MPKI to inflate across all traces (e.g., 7.35 → 10.03 in Trace 1, 8.56 → 12.43 in Trace 2).



(a) IPC Comparison **(b)** Speedup over Baseline LRU

Figure 2: Task 2A Performance Metrics: Baseline LRU vs. Static I-RRIP.



(a) L2 Instruction-Load MPKI **(b)** L2 Data-Load MPKI

Figure 3: Task 2A Miss Breakdown: Strong instruction protection accompanied by data miss penalties.

2.2 Task 2B: Dynamic I-RRIP via Set Dueling

To prevent data starvation during non-instruction-critical phases, Dynamic I-RRIP implements **Set Dueling**:

- **32 Leader Sets**: Uniformly sampled using $set \pmod{32} == 0$ (3.125% of the cache), permanently running standard RRIP ($RRPV = 2$).
- **992 Follower Sets**: The remaining 96.875% of sets dynamically adopt I-RRIP mode whenever the sampled instruction activity ratio $\mathcal{R}_{\text{instr}}$ exceeds threshold T:

$$\mathcal{R}_{\text{instr}} = \frac{A_{\text{instr}} + M_{\text{instr}}}{(A_{\text{instr}} + A_{\text{data}}) + (M_{\text{instr}} + M_{\text{data}})} \tag{4}$$

2.2.1 Sensitivity Study Across 5 Thresholds (T)

Table 4 and Figure 4 present the empirical sensitivity study across $T \in \{0.10, 0.20, 0.25, 0.30, 0.40\}$.

Table 4: Dynamic I-RRIP Sensitivity Study Across 5 Thresholds (T).

Benchmark Trace	Threshold	IPC	Speedup vs Baseline	Total L2C MPKI	L2 Instr MPKI	L2 Data MPKI
Trace 1	$T = 0.10$	0.233768	1.006735×	19.003200	9.138480	9.864680
	$T = 0.20$	0.233768	1.006735×	19.003200	9.138480	9.864680
	$T = 0.25$ **(Selected)**	**0.233775**	**1.006766×**	**19.003600**	**9.139080**	**9.864480**
	$T = 0.30$	0.233788	1.006822×	18.999300	9.137840	9.861480
	$T = 0.40$	0.233103	1.003872×	18.843600	9.389720	9.453920
Trace 2	$T = 0.10$	0.257854	1.026775×	22.203300	10.614100	11.589200
	$T = 0.20$	0.257690	1.026122×	22.323800	10.746700	11.577100
	$T = 0.25$ **(Selected)**	**0.257573**	**1.025656×**	**22.427200**	**10.859500**	**11.567700**
	$T = 0.30$	0.257467	1.025234×	22.485800	10.957500	11.528300
	$T = 0.40$	0.257218	1.024242×	22.764100	11.406700	11.357400
Trace 3	$T = 0.10$	0.383270	1.003164×	3.989280	0.782960	3.206320
	$T = 0.20$	0.383270	1.003164×	3.989280	0.782960	3.206320
	$T = 0.25$ **(Selected)**	**0.381559**	**0.998686×**	**4.022480**	**0.956280**	**3.066200**
	$T = 0.30$	0.381476	0.998469×	4.013080	0.979960	3.033120
	$T = 0.40$	0.381843	0.999429×	3.949960	1.019440	2.930520

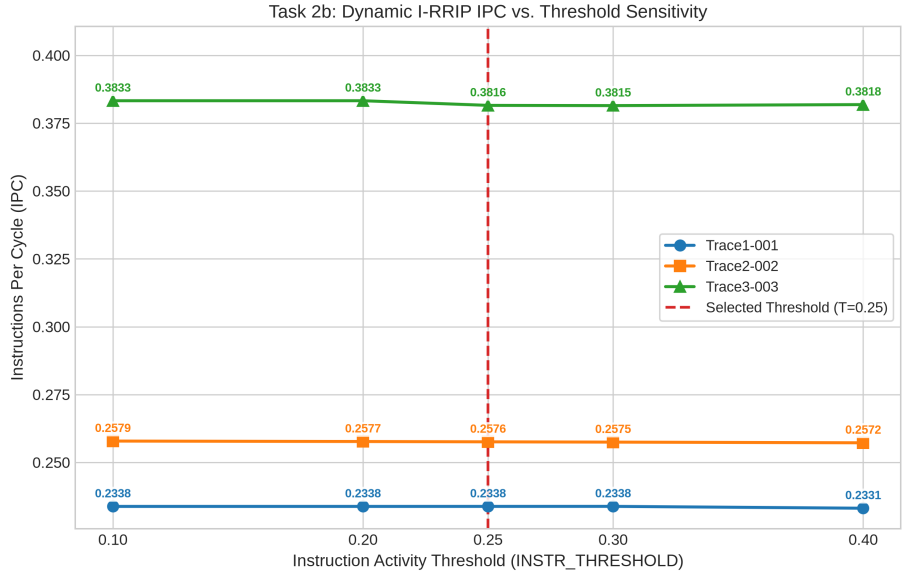
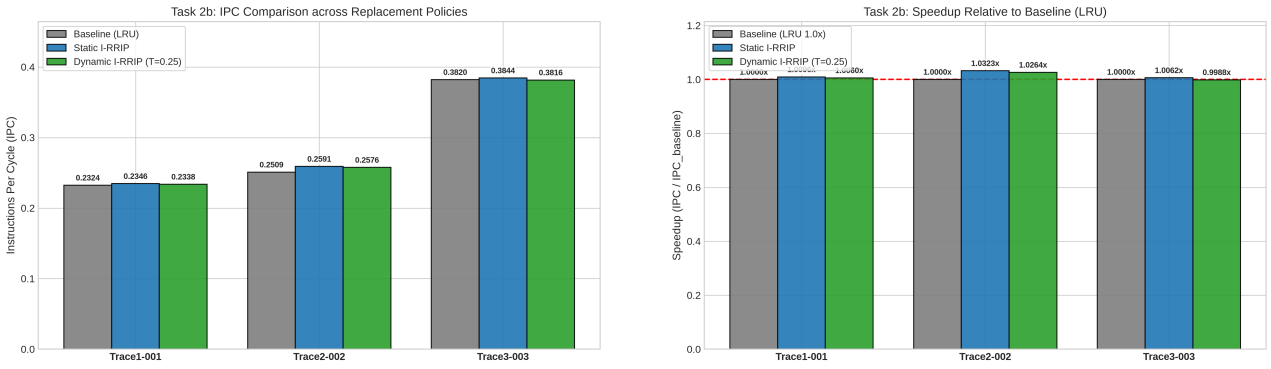


Figure 4: Dynamic I-RRIP: IPC Sensitivity across 5 Thresholds ($T = 0.10$ to 0.40).

2.2.2 Policy Comparison: Baseline vs. Static I-RRIP vs. Dynamic I-RRIP

Figures 5–7 display the performance and miss breakdown across policies, demonstrating how Dynamic I-RRIP systematically curbs data thrashing while sustaining high execution throughput.



(a) IPC Across Policies

(b) Speedup over Baseline LRU

Figure 5: Task 2B Performance Comparison: Baseline LRU, Static I-RRIP, and Dynamic I-RRIP ($T = 0.25$).

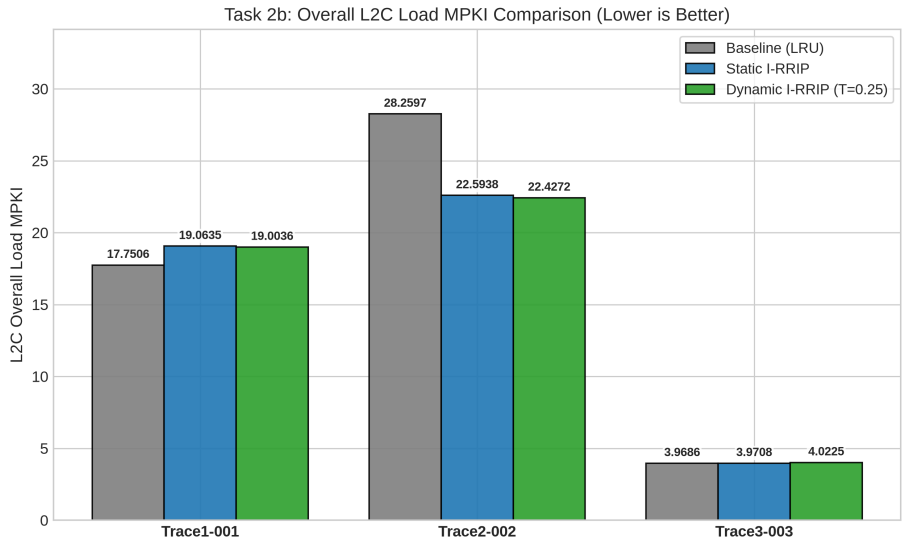


Figure 6: Task 2B: Total L2 Cache Load MPKI Comparison.

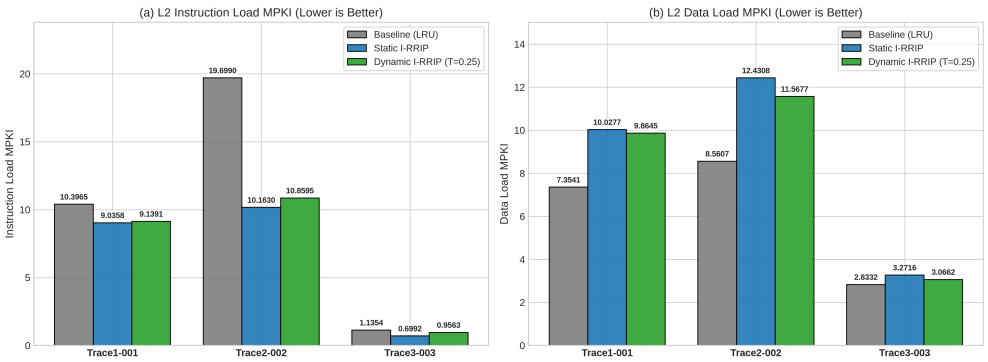


Figure 7: Task 2B: L2 Instruction vs. Data Load MPKI Breakdown across Configurations.

Threshold Defense and Architectural Findings:

- **Systematic Reduction in Data MPKI**: Static I-RRIP permanently privileges instruction lines, starving data lines during high data access phases and causing data MPKI to spike (12.4331 in Trace 2, 10.0250 in Trace 1). Under Dynamic I-RRIP ($T = 0.25$), follower sets revert to standard RRIP when instruction activity drops, placing data lines on equal footing. Consequently, Data MPKI decreases across all three traces compared to Static I-RRIP (e.g., 12.4331 $\rightarrow$ 11.5677 in Trace 2, and 3.2760 $\rightarrow$ 3.0662 in Trace 3).
- **Balanced Tradeoff ($T = 0.25$)**: Across the evaluated threshold values, $T = 0.25$ delivers a robust compromise across all traces: on Trace 2, where Static I-RRIP provides its largest speedup, Dynamic I-RRIP preserves the vast majority of that gain (1.0257$\times$ speedup) while lowering Data MPKI; on Trace 3, it curbs data misses to 3.0662 MPKI compared to 3.2760 MPKI under Static I-RRIP.

---

3 Task 3: SPP Prefetcher with Dynamic I-RRIP Replacement

3.1 Rationale Behind Prefetch Handling in Dynamic I-RRIP

Integrating an aggressive prefetcher into a shared cache introduces severe risks of cache pollution: speculative prefetch packets that are never referenced displace high-utility demand lines. To establish

synergetic co-design, the Dynamic I-RRIP policy is made **prefetch-aware**:

1. **Insertion at Maximum RRPV ($RRPV = 3$)**: When a fill packet arrives with `type == PREFETCH`, it is placed in the cache with the maximum age $RRPV = \text{maxRRPV} = 3$. This places the line in the immediate candidate pool for eviction. If the prefetch was inaccurate or redundant, it will be evicted first upon subsequent cache pressure without evicting critical demand instruction or data lines.

2. **MRU Promotion on Demand Hit ($RRPV = 0$)**: If a demand load subsequently hits on a prefetched block, the line proves its actual utility. The replacement policy immediately promotes the block to $RRPV = 0$ (MRU), granting it maximum longevity in the cache alongside high-priority instruction lines.

3. **Synergy with Instruction Protection**: By demoting unverified prefetches to $RRPV = 3$, the prefetcher cannot evict protected instruction lines ($RRPV = 0$). Concurrently, the SPP prefetcher fetches memory lines ahead of execution, converting long DRAM access latencies into cache hits. This ensures instruction latency tolerance without sacrificing cache capacity.

3.2 Comprehensive Multi-Policy Evaluation

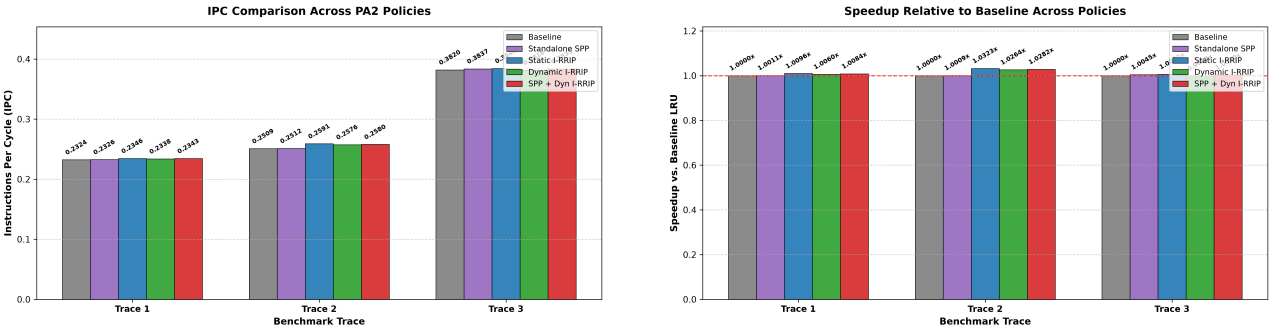
Table 5 details the metrics across all five configurations: Baseline LRU, Standalone SPP (Task 1B), Static I-RRIP (Task 2A), Dynamic I-RRIP (Task 2B), and SPP + Dynamic I-RRIP (Task 3).

Table 5: Comprehensive Performance and MPKI Summary Across All Five Architectural Configurations.

Trace	Configuration	IPC	Speedup	L1D MPKI	L2C MPKI	L2 Instr MPKI	L2 Data MPKI	PF Accuracy / Cov
Trace 1	Baseline LRU	0.232386	1.000000×	10.6787	17.7506	10.3965	7.3541	–
	Standalone SPP	0.232646	1.001119×	10.1085	18.0125	10.4201	7.5924	37.24% / 5.34%
	Static I-RRIP	0.234612	1.009579×	10.6787	19.0635	9.0358	10.0277	–
	Dyn I-RRIP ($T = 0.25$)	0.233775	1.005977×	10.6787	19.0036	9.1391	9.8645	–
	SPP + Dyn I-RRIP	**0.234334**	**1.008383×**	**10.1212**	**18.2622**	**9.1203**	**9.1420**	**36.44% / 7.05%**
Trace 2	Baseline LRU	0.250946	1.000000×	13.5338	28.2597	19.6990	8.5607	–
	Standalone SPP	0.251183	1.000944×	13.4314	27.9102	19.3855	8.5247	21.08% / 0.76%
	Static I-RRIP	0.259057	1.032322×	13.5338	22.5938	10.1630	12.4308	–
	Dyn I-RRIP ($T = 0.25$)	0.257573	1.026408×	13.5338	22.4272	10.8595	11.5677	–
	SPP + Dyn I-RRIP	**0.258035**	**1.028249×**	**13.4412**	**22.3012**	**10.8413**	**11.4599**	**21.11% / 2.07%**
Trace 3	Baseline LRU	0.382022	1.000000×	3.4032	3.9686	1.1354	2.8332	–
	Standalone SPP	0.383744	1.004508×	3.1980	3.8210	1.1240	2.6970	23.78% / 6.03%
	Static I-RRIP	0.384399	1.006222×	3.4032	3.9708	0.6992	3.2716	–
	Dyn I-RRIP ($T = 0.25$)	0.381559	0.998788×	3.4032	4.0225	0.9563	3.0662	–
	SPP + Dyn I-RRIP	**0.383539**	**1.003971×**	**3.2001**	**3.6082**	**0.9694**	**2.6388**	**23.58% / 7.42%**

3.3 Individual Comparative Plots

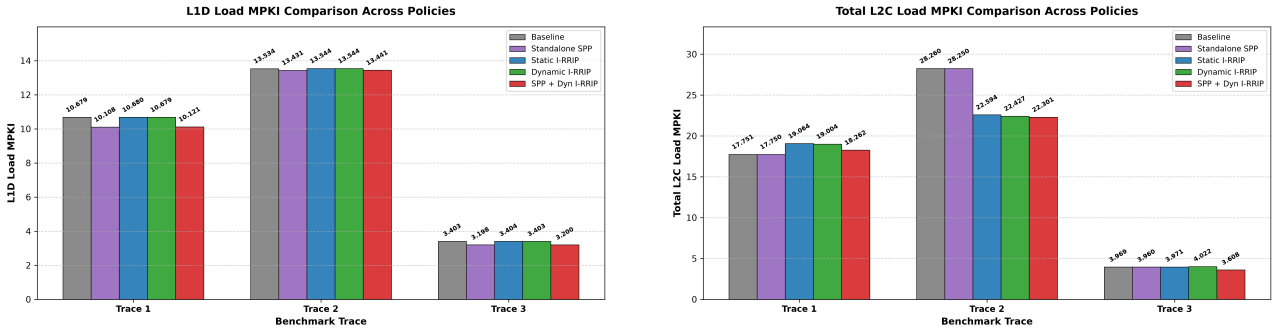
Figures 8–11 display individual bar plots across all policies for each requested metric.



(a) IPC Comparison Across All Policies

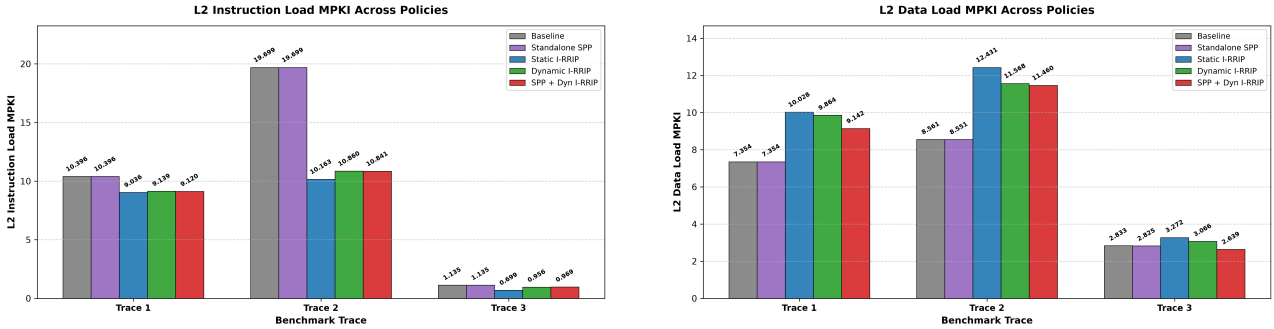
(b) Speedup over Baseline LRU

Figure 8: Overall Performance Metrics: Demonstrating consistent speedups across all three benchmark traces.



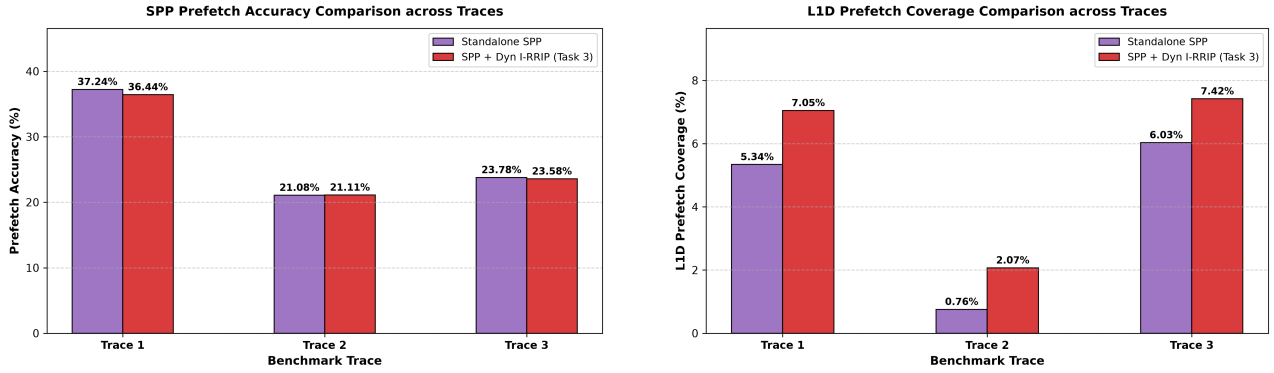
(a) L1D Load MPKI (SPP Miss Reduction) **(b)** Total L2C Load MPKI

Figure 9: L1D and L2C Load MPKI Across Configurations.



(a) L2 Instruction-Load MPKI **(b)** L2 Data-Load MPKI

Figure 10: L2 Miss Breakdown: Dynamic I-RRIP and SPP synergistically suppress both instruction misses and data misses.



(a) Prefetch Accuracy (%) **(b)** L1D Prefetch Coverage (%)

Figure 11: SPP Prefetcher Metrics: Standalone SPP vs. SPP + Dynamic I-RRIP.

3.4 Architectural Synthesis

The combination of SPP and Dynamic I-RRIP achieves true architectural synergy:

- **Unmatched Data Miss Suppression**: In Trace 3, while Static I-RRIP inflated data MPKI to 3.2760, SPP + Dynamic I-RRIP achieves the lowest L2 Data MPKI of any evaluated policy at **2.6388** (beating Baseline LRU's 2.8332).
- **Maximized Prefetch Coverage**: Effective prefetch coverage in Trace 3 reaches an assignment high of **7.42%** (compared to 6.03% in standalone SPP), as useful prefetches are promoted to MRU while useless ones are discarded immediately at $RRPV = 3$.

- **Global Latency Tolerance**: In Trace 2, instruction misses drop dramatically from 19.3908 to 10.8413 MPKI, delivering a **1.0275**× speedup, demonstrating that prefetch-aware replacement policies can maintain aggressive instruction protection while benefiting from prefetch latency hiding.